UE24CS342AA1 – Advanced Algorithms

Advanced Level

Assignment: Fast Polynomial Multiplication using DFT/FFT and RSA

Problem Statement

Design, implement, and experimentally evaluate a system that multiplies two polynomials using the Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT), and demonstrates RSA encryption/decryption of the resulting point-value representation. The implementation must recover the product polynomial using the inverse FFT and validate the result against a direct coefficient-based convolution.

Input

Two polynomials A(x) and B(x), each represented by a coefficient vector of length n. Use the supplied dataset for n = 4, 8, 16, 32, 64, 128. The supplied coefficients are integers in the range 0–9. The vectors are interpreted in ascending coefficient order: A(x) = A[0] + A[1]x + ... + A[n−1]x^(n−1), and similarly for B(x).

Required Implementation – First Five Steps

Step 1 – 1-D DFT using the Vandermonde matrix

Implement the DFT of the coefficient vectors A and B by multiplying each vector by the appropriate DFT/Vandermonde matrix. Record the running time and explain the $O(n^2)$ complexity.

Step 2 – 1-D FFT

Implement the FFT for the same vectors. Compare the FFT output with the DFT output, allowing for small floating-point numerical differences. Record the running time and explain the O(n log n) complexity.

Step 3 – Pointwise multiplication

Multiply corresponding frequency-domain values: $\hat{C}[k] = \hat{A}[k] \times \hat{B}[k]$. This gives the point-value representation of the product polynomial.

Step 4 – RSA encryption and decryption

Demonstrate RSA encryption using public keys generated for 128-bit, 256-bit, and 512-bit modulus sizes, then decrypt using the corresponding private keys and verify recovery of the encoded message. Since FFT values may be complex and RSA operates on integers modulo N,

define and document a reversible encoding for the frequency-domain values (for example, separately quantizing/encoding real and imaginary components). Ensure every encoded integer is smaller than the RSA modulus.

Step 5 – Inverse FFT

Apply the inverse FFT to the decrypted point-value product representation to recover the coefficients of C(x) = A(x)B(x). Round only after establishing a suitable numerical tolerance, and report the recovered coefficient vector.

Correctness Validation

Independently compute the product coefficients using the elementary convolution method: C[k] = Σ(i+j=k) A[i]B[j]. Compare this reference vector with the inverse-FFT result.